

## DESCRIVERE I PL (linguaggi programmazione)

- SINTASSI, grammatica CF e linguaggi regolari
- SEMANTICA, associazione significato-simboli
- PRAGMATICA, buone pratiche *non lo vedremo*
- IMPLEMENTAZIONE
  - interprete
  - compilato
  - soluzione ibrida

## Simple language imperativo

- no dichiarazioni
- solo tipo booleano
- assegnamento e composizione sequenziale
- comando condizionale (if)
- comando iterativo (while, for)

|                                |                                  |
|--------------------------------|----------------------------------|
|                                | PROGRAMMA                        |
| ; controllo                    | comandi composti                 |
| assegnamento                   | comandi base                     |
| op. booleani                   | espressioni                      |
| x...<br>while, if...<br>valori | identificatori,<br>parole chiave |
| ::= ,                          | Notazione della<br>grammatica    |

(⇒) L'operatore tra produzioni

terminali

B-Expr :  $B \rightarrow \text{true} \mid \text{false} \mid (\text{not } B) \mid (B \text{ and } B) \mid (B \text{ or } B)$

↑  
non terminale

comandi atomici

$A \rightarrow x := B \mid \text{skip}$

comandi composti

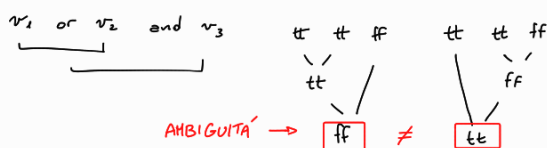
$C \rightarrow A \mid C ; C \mid \text{if } B \text{ then } C \text{ else } C \mid$   
 $\text{while } B \text{ do } C$

programma

$P \rightarrow C$

senza  
parentesi

$B \rightarrow \text{true} \mid \text{false} \mid \text{not } B \mid B \text{ and } B \mid B \text{ or } B$



Sintassi Semantica

analisi semantica  
semantica statica

↳ n. parametri attuali in una chiamata a procedura

↳ correttezza dei tipi

⇒ vincoli sintattici

SEMANTICA → associazione di "significati" ai simboli di linguaggio, e di conseguenza agli elementi del linguaggio

- **semantica denotazionale** descriviamo funzionalità del programma  
(comportamento I/O) ⇒ COSA calcola

- **semantica assiomatica** descrive il comportamento dimostrando proprietà  
⇒ COSA soddisfa il calcolo

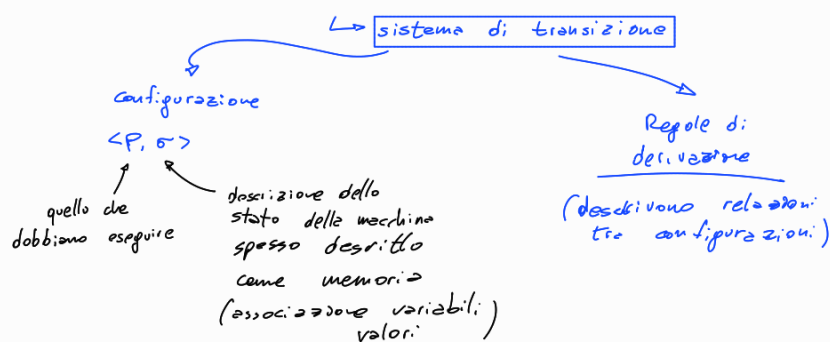
- **semantica operativa** descrive il comportamento mediante trasformazioni di stato  
⇒ COME calcola

$$P \begin{cases} z := z; \\ y := z; \\ y := y+1; \\ z := y; \end{cases} \quad \text{Semantica denotazionale} \rightarrow I/O$$

$$S_0 = \langle z \mapsto 1, y \mapsto 1 \rangle$$
 stato iniziale  
 con  $z$  e  $y$  non  
 definiti ( $\perp$ )

$$\begin{aligned}
 \mathcal{D}(P)(S_0) &= [\mathcal{D}(z:=y) \circ \mathcal{D}(y:=y+1) \circ \mathcal{D}(y:=z) \circ \mathcal{D}(z:=z)](S_0) \\
 &\quad \text{composizione delle semantiche} \\
 &\quad \text{dei simboli assegnamenti} \\
 &= \mathcal{D}(z:=y) \circ \mathcal{D}(y:=y+1) \circ \mathcal{D}(y:=z) \langle z \mapsto 2, y \mapsto 1 \rangle \\
 &\quad \langle z \mapsto 2, y \mapsto 2 \rangle \\
 &= \mathcal{D}(z:=y) \circ \mathcal{D}(y:=y+1) \langle z \mapsto 2, y \mapsto 2 \rangle \\
 &\quad \langle z \mapsto 2, y \mapsto 3 \rangle \\
 &= \mathcal{D}(z:=y) \langle z \mapsto 2, y \mapsto 3 \rangle = \langle z \mapsto 3, y \mapsto 3 \rangle \\
 &\quad \langle z \mapsto 3, y \mapsto 3 \rangle
 \end{aligned}$$

Semantica operativa descrive i passi di calcolo che portano dall'input all'output



$$\begin{aligned}
 P \begin{cases} z := z \\ y := z \\ y := y+1 \\ z := y \end{cases} &\rightarrow \begin{aligned} &\langle P, \langle z \mapsto 1, y \mapsto 1 \rangle \rangle \\ &\rightarrow \langle y := z; y := y+1; z := y, \langle z \mapsto 2, y \mapsto 1 \rangle \rangle \\ &\rightarrow \langle y := y+1; z := y, \langle z \mapsto 2, y \mapsto 2 \rangle \rangle \\ &\rightarrow \langle z := y, \langle z \mapsto 2, y \mapsto 3 \rangle \rangle \\ &\rightarrow \langle E, \langle z \mapsto 3, y \mapsto 3 \rangle \rangle \end{aligned}
 \end{aligned}$$

Semantica  $\rightarrow$  traccia di configurazioni sequenza

$$\langle P, \sigma \rangle \xrightarrow{I/O} \langle P', \sigma' \rangle \rightarrow \langle P'', \sigma'' \rangle \rightarrow \langle P''', \sigma''' \rangle \rightarrow \langle E, \sigma_f \rangle$$

## INDUZIONE STRUTTURALE

Dimostrazione per induzione strutturale di una proprietà  $\pi$

$\hookrightarrow$  dimostro  $\pi$  per tutti gli elementi base della categoria  
 (quelli generati in produzione in cui o da non comparire il non terminale della categoria)

$B \rightarrow \text{true}$   $B \rightarrow \text{false}$   $\rightarrow$  elementi base  
 $B \rightarrow (\text{not } B)$   $\rightarrow$  non è un elemento base

$\hookrightarrow$  dimostro  $\pi$  per tutti gli elementi composti della categoria  
 (quelli generati da produzioni in cui il simbolo della categoria è  $\Box$ )

$B \rightarrow (\text{not } B)$   $(B \text{ and } B)$   $(B \text{ or } B)$   
 elemento composto

Lo assumendo che  $\pi$  valga per tutti i componenti immediati (gli elementi della categoria a dx della produzione)

$$B \rightarrow (\text{not } \underline{B}) \mid (\underline{B} \text{ and } \underline{B}) \mid (\underline{B} \text{ or } \underline{B})$$

componenti immediati

### Esempi

Definiamo  $x \in \mathbb{N}$  per induzione strutturale

Base:  $0 \in x$

Passo:  $n \in x \Rightarrow n+3 \in x$

Definizione costruttiva

multiplo di 3

Dimostriamo che  $x = \{n \mid n \in 3\mathbb{N}\}$

Base

Elemento base:  $0 \in 3\mathbb{N}$

Passo

Prendiamo  $n \in x$

Hp. ind. supponiamo  $n \in 3\mathbb{N}$   
dobbiamo dimostrare  
 $n+3 \in 3\mathbb{N}$

$n \in 3\mathbb{N}$  per def.  $\exists i. n = 3i$

$n+3 = 3i+3 = 3(i+1) \in 3\mathbb{N}$  per def.

Definiamo Alberi Binari (BT)

$N$  = insieme dei nodi

Base se  $n \in N$  allora  $n \in BT$

$N \subseteq BT$

Passo  $T_1, T_2 \in BT$   $br(n, T_1, T_2) \leftarrow$  specifichiamo la radice  
 $n \in N$   $br(T_1, T_2) \leftarrow \forall n \in N$

